

Lab 3: Architectural Style Justification

Problem Statement #04 — Student Project Portfolio & Showcase App

Architecture Selection: We chose Microservices Architecture for the Student Project Portfolio & Showcase App.

Architectural Choice

The system is decomposed into six independently deployable services — Profile & Submission Service, Media Storage Service, Judging Service, Voting Service, Anti-Sybil Verification Service, and Leaderboard Service — communicating through well-defined provided/required interfaces (see accompanying Component Diagram).

Two Scenario-Related Reasons

- Independent scaling under uneven load: during a live expo, public voting traffic can spike far beyond profile-browsing or judging traffic (e.g., hundreds of simultaneous votes right after a demo). A microservices split lets the Voting Service and Anti-Sybil Verification Service be scaled out independently to meet the sub-300ms latency target (NFR-001), without over-provisioning the Judging or Media Storage services, which see much lower and steadier traffic.
- Fault isolation across unrelated concerns: artifact uploads (large video/image files), rubric scoring, and public voting have very different failure modes and change frequencies. If the Media Storage Service is slow under a burst of video uploads, that must not block or crash the Voting Service or Leaderboard Service, which need to stay responsive throughout the event. Isolating these into separate services contains failures to a single capability instead of taking down the whole showcase.

Security Advantage

Isolating anti-sybil logic into its own Anti-Sybil Verification Service lets sensitive verification data (device fingerprints, OTP state, failed-attempt counters) be secured and rate-limited behind a single, narrowly-scoped internal API (AntiSybilCheckAPI) that only the Voting Service can call. This keeps the anti-sybil rules and audit logs centralized and independently hardened, rather than duplicated (and potentially inconsistently enforced) inside every service that touches votes, directly satisfying the vote-integrity requirement in NFR-002.

Performance Benefit

Because the Voting Service and Leaderboard Service are separate, lightweight services dedicated solely to recording votes and recalculating rankings, they can be optimized (in-memory aggregation, horizontal replicas, a fast cache in front of LeaderboardQueryAPI) specifically to hit the sub-300ms real-time update target in NFR-001 — without that performance-critical hot path being slowed down by heavier, less latency-sensitive operations such as media file storage or multi-criteria rubric grading.

Component Diagram - Student Project Portfolio and Showcase App (Microservices Architecture)

